

Agentic SRA Framework — Software Requirements Specification (SRS)

Document: SRS-FW-001 · **Version:** 1.0 · **Status:** Approved (2026-09-10) **Traces up to:** docs/use_cases.md v1.0 **Audience:** Cybersecurity engineers, regulatory reviewers, capstone advisor

1. Purpose and scope

This SRS defines what the Agentic SRA Framework shall do, at the level of behavior that can be verified by a test. Every requirement traces up to at least one use case in docs/use_cases.md . The next level down — how each subsystem is designed to satisfy these requirements — is documented in docs/architecture.md (SAD, Phase 3) and docs/detail_design.md (SDD, Phase 4).

2. Conventions

- **Shall** — a mandatory behavior. Failure to meet a shall-statement is a defect.
- **Should** — a strongly preferred behavior; deviations are recorded and reviewed.
- **May** — a permitted behavior; absence is not a defect.
- **Priority** — High (v1 blocker), Medium (v1 target), Low (nice-to-have, v1 optional).
- **Verify** — the primary verification method (Unit test / Integration test / System test / Manual review / Code review).

3. Functional requirements

3.1 SRS · CLI — command-line interface

SRS-CLI-0001 · Framework entry point The framework shall provide a single command-line entry point sra installed via pip install -e . . *Priority:* High · *Traces to:* UC-FW-01, UC-FW-02, UC-FW-03, UC-FW-04, UC-FW-05, UC-FW-06 · *Verify:* System test

SRS-CLI-0002 · Subcommand set The `sra` CLI shall expose these subcommands: `init`, `status`, `extract`, `index`, `mcp`, `mcp-test`, `draft`, `review`, `export`, `refresh`. *Priority: High* · *Traces to: UC-FW-01..07* · *Verify: System test*

SRS-CLI-0003 · `sra init` clones a corpus `sra init --repo <URL> --device <name>` shall git-clone the URL into `input/<device>/` and write a manifest capturing commit SHA, commit subject, clone timestamp, file count, and total size. *Priority: High* · *Traces to: UC-FW-01* · *Verify: System test*

SRS-CLI-0004 · `sra init` refuses to overwrite `sra init` shall refuse to run when `input/<device>/` is non-empty unless `--force` is passed. *Priority: High* · *Traces to: UC-FW-01/A1* · *Verify: Integration test*

SRS-CLI-0005 · `sra init` supports pinned commit `sra init` shall accept `--commit <SHA>` to check out a specific commit after cloning, enabling reproducible re-runs required by the audit trail. *Priority: High* · *Traces to: UC-FW-07, SRS-NFR-0009* · *Verify: Integration test*

SRS-CLI-0006 · `sra status` reports state `sra status --device <name>` shall print the manifest and indicate whether vector, extracts, and graph stores are present and up-to-date relative to the current corpus commit. *Priority: High* · *Traces to: UC-FW-01* · *Verify: System test*

SRS-CLI-0007 · `sra index` builds all indices `sra index --device <name>` shall build vector, extracts, and graph in order and exit non-zero if any stage fails. *Priority: High* · *Traces to: UC-FW-01* · *Verify: System test*

SRS-CLI-0008 · `sra index` supports stage flags `sra index` shall accept `--no-vectors`, `--no-extract`, `--no-graph`, and `--dry-run` to skip stages or count-only. *Priority: Medium* · *Traces to: UC-FW-01* · *Verify: Integration test*

SRS-CLI-0009 · `sra draft` runs the agent pipeline `sra draft --device <name>` shall spawn the MCP servers, run the LangGraph agent pipeline, and write incremental progress to `output/<device>_sra_draft.md` and `.json`. *Priority: High* · *Traces to: UC-FW-02* · *Verify: System test*

SRS-CLI-0010 · `sra draft` supports scoping `sra draft` shall accept `--limit N`, `--category <cat>`, and `--threat <id>` to restrict the draft to a subset of threats. *Priority: Medium* · *Traces to: UC-FW-02/A1* · *Verify: Integration test*

SRS-CLI-0016 · `sra draft --profile` selects the model strategy `sra draft` shall accept `--profile {free|openai|hybrid}` (default `hybrid`) that selects the per-agent model plan defined in SRS-AGT-0011. The chosen profile shall be recorded in the provenance file. *Priority: High* · *Traces to: UC-FW-02* · *Verify: Integration test*

SRS-CLI-0011 · `sra review` launches the UI `sra review --device <name>` shall start the Streamlit review UI on `http://localhost:8501` and open the default browser to it. *Priority: High* · *Traces to: UC-FW-03* · *Verify: Manual review*

SRS-CLI-0012 · `sra export` produces four formats `sra export --device <name>` shall write `<device>_sra_final.md`, `.pdf`, `.docx`, and `.json` under `output/`, plus `<device>_provenance.json` and `<device>_prompts.jsonl`. *Priority: High* · *Traces to: UC-FW-04, UC-FW-07* · *Verify: System test*

SRS-CLI-0013 · `sra refresh` handles corpus updates `sra refresh --device <name>` shall `git pull` the corpus, rebuild indices, detect affected threats, re-run only those agents, and write `<device>_sra_delta.md`. *Priority: Medium* · *Traces to: UC-FW-05* · *Verify: System test*

SRS-CLI-0014 · `sra mcp` launches servers `sra mcp {vector|graph|files} --device <name>` shall launch the named MCP server on stdio for a client to connect. *Priority: High* · *Traces to: UC-FW-06* · *Verify: Integration test*

SRS-CLI-0015 · `sra mcp-test` smoke-tests all servers `sra mcp-test --device <name>` shall call every MCP tool in-process and print a pass/fail summary. *Priority: Medium* · *Traces to: UC-FW-06* · *Verify: Integration test*

3.2 SRS · IDX — indexing pipeline

SRS-IDX-0001 · Vector store choice The vector store shall be Chroma with a persistent client at `stores/chroma/`. *Priority: High* · *Traces to: UC-FW-01* · *Verify: Code review*

SRS-IDX-0002 · Embedding model Embeddings shall be produced by OpenAI `text-embedding-3-large` (3072 dims) by default. The embedder shall be swappable via a single `Embedder` class. *Priority: High* · *Traces to: UC-FW-01* · *Verify: Unit test*

SRS-IDX-0003 · Collections Chroma shall hold three collections: `device_docs` (SRS/SDD/SDS/SAD/threats/vulns/narrative), `device_code` (firmware functions), `regulatory` (FDA/IEC/AAMI/NIST clauses). *Priority: High* · *Traces to: UC-FW-01* · *Verify: Integration test*

SRS-IDX-0004 · Chunk-level metadata Every chunk shall carry metadata including `source`, `kind`, `item_id` (where applicable), `category`, `subsystem`, and any layer-specific traceability fields (`traces_to`, `implements`, `realizes`, `location`). *Priority: High* · *Traces to: UC-FW-02, UC-FW-06* · *Verify: Unit test*

SRS-IDX-0005 · Deterministic chunk ids Chunk ids shall be deterministic across runs so that re-indexing is idempotent. Collisions shall be resolved by suffixing `#2`, `#3`, *Priority: High* · *Traces to: UC-FW-01* · *Verify: Unit test*

SRS-IDX-0006 · Embedding cost cap The embedder shall report per-batch and cumulative estimated cost and shall abort with a clear error if the cumulative cost exceeds a `SRA_EMBED_BUDGET_USD` environment cap when set. *Priority: Medium · Traces to: UC-FW-01 · Verify: Integration test*

SRS-IDX-0007 · Graph extractor produces 7 JSON files The extractor shall produce `requirements.json`, `threats.json`, `dfd_elements.json`, `controls.json`, `vulnerabilities.json`, `code_artifacts.json`, `use_cases.json` under `stores/extracts/<device>/`. *Priority: High · Traces to: UC-FW-01 · Verify: Integration test*

SRS-IDX-0008 · Graph schema The graph shall define 11 node tables (Requirement, DesignItem, ImplementationItem, ArchitectureItem, Threat, Vulnerability, CodeArtifact, Control, UseCase, Actor, DFDElement) and 8 relationship tables (TRACES_TO, IMPLEMENTS, REALIZES, REALIZED_BY, LOCATED_IN, MITIGATES, APPLIES_TO, INTERACTS_VIA). *Priority: High · Traces to: UC-FW-01, UC-FW-06 · Verify: Code review*

SRS-IDX-0009 · Dangling edges skipped and reported The graph loader shall skip any edge whose source or target id is absent from the node set, count the skips per relationship type, and write the raw skipped edges to `_dangling_edges.json` for later review. *Priority: High · Traces to: UC-FW-01/A3, UC-FW-02 · Verify: Integration test*

SRS-IDX-0010 · Graph load is idempotent Re-running `sra index --graph` shall produce a database that responds identically to Cypher queries as the prior run for the same corpus commit. *Priority: High · Traces to: UC-FW-05 · Verify: Integration test*

3.3 SRS · MCP — Model Context Protocol servers

SRS-MCP-0001 · corpus-vector tool set The `corpus-vector` MCP server shall expose the tools `search(query, k, filters, collection)`, `get_chunk(chunk_id, collection)`, and `list_collections()`. *Priority: High · Traces to: UC-FW-02, UC-FW-06 · Verify: Integration test*

SRS-MCP-0002 · corpus-graph tool set The `corpus-graph` MCP server shall expose `cypher(query, params)`, `neighbors(node_id, edge_type, direction, depth)`, `path(from_id, to_id, max_depth)`, and `list_schema()`. *Priority: High · Traces to: UC-FW-02, UC-FW-06 · Verify: Integration test*

SRS-MCP-0003 · corpus-graph is read-only `corpus-graph.cypher` shall reject any query containing CREATE, MERGE, DELETE, SET, DROP, ALTER, COPY, INSERT, REMOVE, or LOAD, in any case. *Priority: High · Traces to: UC-FW-06 · Verify: Unit test*

SRS-MCP-0004 · corpus-files tool set The `corpus-files` MCP server shall expose `read(path)`, `list_dir(directory)`, and `grep(pattern, glob, max_hits, case_insensitive, context)`. *Priority: High · Traces to: UC-FW-02, UC-FW-06 · Verify: Integration test*

SRS-MCP-0005 · corpus-files refuses path traversal `corpus-files.read` and `list_dir` shall refuse any path that resolves outside `input/<device>/`, including via `..` segments or absolute paths. *Priority: High · Traces to: UC-FW-06 · Verify: Unit test*

SRS-MCP-0006 · corpus-files caps reads `corpus-files.read` shall return at most 200 KB of content and set a `truncated` flag when the file exceeds that limit. *Priority: Medium · Traces to: UC-FW-06 · Verify: Unit test*

SRS-MCP-0007 · Servers run on stdio Each MCP server shall run on stdio transport when launched via `sra mcp <server>`, matching the MCP 2.x transport contract so any conformant client can connect. *Priority: High · Traces to: UC-FW-02, UC-FW-06 · Verify: Integration test*

3.4 SRS · AGT — five agents

SRS-AGT-0001 · Ingestion agent The Ingestion agent shall enumerate threats from the graph (all threats or a filtered subset per `--limit` / `--category` / `--threat`) and feed one threat per iteration to the downstream agents. *Priority: High · Traces to: UC-FW-02 · Verify: Integration test*

SRS-AGT-0002 · Compliance Mapper agent The Compliance Mapper agent shall, for each threat, retrieve the relevant regulatory clauses via `corpus-vector.search` and walk `Control-[:SATISFIES]-> Clause` edges via `corpus-graph.cypher`, and decide a verdict (`present` / `partial` / `missing`) per clause with a short evidence citation. *Priority: High · Traces to: UC-FW-02 · Verify: System test*

SRS-AGT-0003 · Code Analysis agent The Code Analysis agent shall use the graph to find implicated code artifacts for the threat, use `corpus-files.grep` and `corpus-files.read` to confirm or refute the presence of the expected control in code, and produce a finding (`present` / `partial` / `missing`) with file and line citations. *Priority: High · Traces to: UC-FW-02 · Verify: System test*

SRS-AGT-0004 · Threat / Control / Risk agent The Threat/Control/Risk agent shall compose a 12-field SRA entry per threat with: `threat_id`, `title`, `stride`, `assets_affected`, `existing_controls`, `gap_analysis` (per clause), `cvss_v31` (vector string + base + temporal + environmental), `residual_risk`, `proposed_controls`, `proposed_doc_changes` (file:section), `proposed_code_changes` (file:line hunks), `priority` + `effort`. *Priority: High · Traces to: UC-FW-02 · Verify: System test*

SRS-AGT-0005 · Report Generator agent The Report Generator agent shall write each finalized SRA entry incrementally to `output/<device>_sra_draft.md` and `.json` so a crash mid-run keeps every completed entry. *Priority:* High · *Traces to:* UC-FW-02 · *Verify:* Integration test

SRS-AGT-0006 · LiteLLM abstraction Every agent LLM call shall go through LiteLLM, keyed by a model id in `config.yaml`. Swapping models shall not require agent code changes. *Priority:* High · *Traces to:* UC-FW-02 · *Verify:* Code review

SRS-AGT-0007 · Retry and cross-provider fallback Agent LLM calls shall retry with exponential backoff up to 3 attempts on 429 / rate-limit / timeout errors. If retries are exhausted on the primary provider (Groq or Gemini), the call shall automatically fall back to OpenAI `gpt-4o-mini` and mark the entry with `fallback_used: <original>-openai:gpt-4o-mini` in the provenance record. *Priority:* High · *Traces to:* UC-FW-02/A2 · *Verify:* Integration test with simulated 429

SRS-AGT-0008 · Every LLM turn is logged Every LLM call shall append one JSON object to `output/<device>_prompts.jsonl` with `entry_id`, `agent`, `model`, `prompt`, `tool_calls`, `response`, `latency_ms`, `tokens_in`, `tokens_out`, `timestamp_utc`. *Priority:* High · *Traces to:* UC-FW-07 · *Verify:* Integration test

SRS-AGT-0009 · Parse-error handling An SRA entry whose LLM output fails schema validation shall be written with a `parse_error` flag and the raw LLM response preserved for manual correction, rather than being dropped. *Priority:* High · *Traces to:* UC-FW-02/A3 · *Verify:* Unit test

SRS-AGT-0010 · CVSS scoring rubric The Threat/Control/Risk agent shall follow CVSS v3.1 base metrics (Attack Vector, Attack Complexity, Privileges Required, User Interaction, Scope, Confidentiality, Integrity, Availability) plus temporal and environmental metrics tailored to the medical-device context (see the CVSS rubric in `docs/detail_design.md`). *Priority:* High · *Traces to:* UC-FW-02 · *Verify:* Manual review vs seeded ground truth

SRS-AGT-0011 · Per-agent model plan by profile Model selection shall be driven by the `--profile` flag on `sra draft` (SRS-CLI-0016). The mapping shall be:

Agent	free profile	hybrid profile (default)	openai profile
Ingestion	<code>groq/openai/gpt-oss-120b</code>	<code>groq/openai/gpt-oss-120b</code>	<code>openai/gpt-4o-mini</code>
Compliance Mapper	<code>gemini/gemini-2.5-flash</code>	<code>gemini/gemini-2.5-flash</code>	<code>openai/gpt-4o-mini</code>
Code Analysis	<code>groq/openai/gpt-oss-120b</code>	<code>groq/openai/gpt-oss-120b</code>	<code>openai/gpt-4o-mini</code>

Agent	free profile	hybrid profile (default)	openai profile
Threat / Control / Risk	groq/openai/gpt-oss-120b	openai/gpt-4o	openai/gpt-4o
Report Generator	groq/openai/gpt-oss-120b	groq/openai/gpt-oss-120b	openai/gpt-4o-mini
Embeddings (build-time)	openai/text-embedding-3-large	same	same

Cross-provider fallback per SRS-AGT-0007 applies to every non-OpenAI slot. *Priority*: High · *Traces to*: UC-FW-02, SRS-NFR-0002, SRS-NFR-0005, SRS-NFR-0011 · *Verify*: Integration test per profile

3.5 SRS · UI — review interface

SRS-UI-0001 · Streamlit port The review UI shall start on `http://localhost:8501` by default, with a `--port` override. *Priority*: High · *Traces to*: UC-FW-03 · *Verify*: Manual review

SRS-UI-0002 · Per-entry cards The UI shall render one card per SRA entry showing threat title + STRIDE, CVSS, existing controls, gap analysis, proposed controls, proposed doc changes, proposed code diffs, and action buttons **Approve / Edit / Reject / Defer**. *Priority*: High · *Traces to*: UC-FW-03 · *Verify*: Manual review

SRS-UI-0003 · Edit inline Selecting **Edit** shall expose every field of the SRA entry as an inline editor. Saved edits shall replace the draft entry, and the delta shall be recorded in the audit log. *Priority*: High · *Traces to*: UC-FW-03 · *Verify*: Manual review

SRS-UI-0004 · Reject requires reason Selecting **Reject** shall require a free-text reason before saving. Rejected entries shall be excluded from the final SRA but retained in the audit log. *Priority*: High · *Traces to*: UC-FW-03 · *Verify*: Manual review

SRS-UI-0005 · State persistence Review state (which entries reviewed, decisions, reviewer identity, timestamps) shall be persisted to `output/<device>_review_state.json` on every action so the UI can be paused and resumed without loss. *Priority*: High · *Traces to*: UC-FW-03 · *Verify*: Integration test

SRS-UI-0006 · Ask-the-graph action The UI shall provide an **Ask the graph** action on each card that runs a live Cypher query via `corpus-graph` and shows the result without leaving the card. *Priority*: Medium · *Traces to*: UC-FW-03, UC-FW-06 · *Verify*: Manual review

SRS-UI-0007 · Progress indicator The UI shall display running progress (X of N threats reviewed) and a breakdown by disposition (approved / edited / rejected / deferred). *Priority:* Medium · *Traces to:* UC-FW-03 · *Verify:* Manual review

3.6 SRS · OUT — output artifacts

SRS-OUT-0001 · Draft file The draft SRA shall be written to `output/<device>_sra_draft.md` (human) and `output/<device>_sra_draft.json` (machine). *Priority:* High · *Traces to:* UC-FW-02 · *Verify:* System test

SRS-OUT-0002 · Final file After review, `sra export` shall write `output/<device>_sra_final.md`, `.pdf`, `.docx`, and `.json`. *Priority:* High · *Traces to:* UC-FW-04 · *Verify:* System test

SRS-OUT-0003 · PDF rendering pipeline PDF rendering shall use pandoc (markdown → HTML) plus Chrome headless (HTML → PDF), to avoid the macOS-SIP incompatibility with weasyprint/pango. *Priority:* High · *Traces to:* UC-FW-04 · *Verify:* System test

SRS-OUT-0004 · DOCX rendering DOCX rendering shall use pandoc directly. The DOCX shall preserve table structure, headings, and code blocks suitable for review with Word change-tracking. *Priority:* High · *Traces to:* UC-FW-04 · *Verify:* Manual review

SRS-OUT-0005 · Provenance file `output/<device>_provenance.json` shall record: corpus commit SHA, framework version, per-agent model ids, per-agent token counts, per-entry review decisions, reviewer identities, timestamps. *Priority:* High · *Traces to:* UC-FW-07 · *Verify:* Integration test

SRS-OUT-0006 · Prompts log `output/<device>_prompts.jsonl` shall record every LLM turn as a JSON line per SRS-AGT-0008, enabling exact reasoning replay. *Priority:* High · *Traces to:* UC-FW-07 · *Verify:* Integration test

SRS-OUT-0007 · Every SRA claim is cited Every field in every SRA entry that references corpus content shall include a citation of the form `<source>:<item_id_or_line>` (e.g. `SRS.md:SRS-0TA-0014`, `install_handler.c:82`). *Priority:* High · *Traces to:* UC-FW-07 · *Verify:* Unit test on output JSON

4. Non-functional requirements

4.1 SRS · NFR — cost, performance, reproducibility, portability, security

SRS-NFR-0001 · Embedding cost cap Building the vector index for a corpus of ≤ 5000 chunks shall cost $\leq \$1$ USD in OpenAI charges at `text-embedding-3-large` list price (2026). *Priority:* High · *Traces to:* UC-FW-01 · *Verify:* Metric measured in `sra index` output; asserted in test.

SRS-NFR-0002 · Draft cost cap by profile A full SRA draft for a corpus of ≤ 300 threats shall cost:

- `free` profile: **\$0** in agent LLM charges (Groq + Gemini free-tier only, plus fallback to OpenAI `gpt-4o-mini` only if free tier fails — spillover $\leq \$1$)
- `hybrid` profile: **$\leq \$5$** in agent LLM charges (OpenAI `gpt-4o` only on the Threat/Control/Risk agent; free tier for the other four)
- `openai` profile: **$\leq \$15$** in agent LLM charges (`gpt-4o` on TCR, `gpt-4o-mini` on the other four) *Priority:* High · *Traces to:* UC-FW-02 · *Verify:* Metric measured in provenance per profile; asserted in test.

SRS-NFR-0003 · Draft latency `sra draft` shall complete for a corpus of 225 threats in ≤ 30 minutes on a laptop with a stable network, using default models. *Priority:* Medium · *Traces to:* UC-FW-02 · *Verify:* System test with timing.

SRS-NFR-0004 · Reproducibility Running `sra draft` twice on the same corpus commit with the same model ids and `temperature=0` shall produce SRA entries that agree on all structured fields (`threat_id`, `cvss_v31`, `residual_risk`, `priority`) and semantically-equivalent free-text fields. *Priority:* High · *Traces to:* UC-FW-07 · *Verify:* Integration test.

SRS-NFR-0005 · Provider portability The framework shall support at minimum OpenAI (paid) plus Groq and Gemini (free tier) via LiteLLM. Model selection is driven by the `--profile` flag (SRS-CLI-0016) mapping to the plan in SRS-AGT-0011. Adding another provider shall require only a config-file edit, no code changes. *Priority:* High · *Traces to:* UC-FW-02 · *Verify:* Integration test running each profile once.

SRS-NFR-0006 · Local-first data plane No corpus content, firmware source, or PHI shall leave the local machine except via explicit outbound LLM calls (embeddings + agent turns). All stores shall live on local disk. *Priority:* High · *Traces to:* UC-FW-01, UC-FW-06 · *Verify:* Code review; egress test.

SRS-NFR-0007 · Secrets handling API keys shall be loaded only from `.env` at framework startup and never logged. `.gitignore` shall exclude `.env`. *Priority:* High · *Traces to:* UC-FW-01 · *Verify:* Code review.

SRS-NFR-0008 · OS support The framework shall run on macOS 13+ and Linux (Ubuntu 22.04+, Debian 12+) with Python 3.10 or newer. *Priority:* High · *Traces to:* UC-FW-01 · *Verify:* CI on both OS.

SRS-NFR-0009 · Auditability Every SRA claim in the final file shall be traceable to (a) a corpus item at a specific commit SHA, (b) the exact LLM prompt that produced the claim, and (c) a reviewer decision by a named person at a specific timestamp. *Priority:* High · *Traces to:* UC-FW-07 · *Verify:* Integration test on provenance completeness.

SRS-NFR-0010 · Idempotent re-runs Re-running any of `sra init`, `sra index`, `sra draft` on the same inputs shall produce equivalent stores/outputs (modulo LLM sampling variance covered by SRS-NFR-0004) without side effects on unrelated files. *Priority:* High · *Traces to:* UC-FW-01, UC-FW-05 · *Verify:* Integration test.

SRS-NFR-0011 · Free-tier operable The framework shall be operable end-to-end using the `free` profile (Groq + Gemini free tiers) plus one-time OpenAI embedding spend ($\leq \$1$ for the PMx-100 corpus). The `free` profile shall be the demo default in `README.md` for capstone evaluation. *Priority:* High · *Traces to:* UC-FW-01, UC-FW-02, SRS-CLI-0016 · *Verify:* Demo run using only free-tier keys plus OpenAI for embeddings.

SRS-NFR-0012 · Human-in-the-loop by default The framework shall not mark an SRA entry as `final` without an explicit reviewer decision. There shall be no `--auto-approve-all` flag. *Priority:* High · *Traces to:* UC-FW-03, UC-FW-04 · *Verify:* Code review; missing-flag test.

SRS-NFR-0013 · Graceful degradation on partial corpus Missing or malformed input files shall be logged and skipped rather than aborting the run. The affected SRA entries shall carry a `data_quality: gap` flag pointing at what was missing. *Priority:* Medium · *Traces to:* UC-FW-01/A3, UC-FW-02 · *Verify:* Integration test with a corrupted corpus.

SRS-NFR-0014 · Open licenses only Every third-party dependency shall be under an OSI-approved license compatible with MIT redistribution. *Priority:* High · *Traces to:* — · *Verify:* License audit.

SRS-NFR-0015 · Documentation The framework shall ship with `README.md`, `docs/use_cases.md`, `docs/requirements.md`, `docs/architecture.md`, and `docs/detail_design.md`, plus a one-command demo pointed at the PMx-100 reference corpus. *Priority:* High · *Traces to:* — · *Verify:* File existence + link check.

5. Traceability matrix

Every requirement traces to at least one use case in `docs/use_cases.md` v1.0. Coverage summary (rows = UC, columns = requirement categories):

Use case	CLI	IDX	MCP	AGT	UI	OUT	NFR
UC-FW-01 Onboard corpus	0001- 0008	0001- 0010	—	—	—	—	0001, 0006- 0011, 0013- 0015
UC-FW-02 Draft SRA	0001, 0002, 0009- 0010	0004	0001- 0002, 0004, 0007	0001- 0010	—	0001	0002- 0005, 0011- 0013
UC-FW-03 Review	0001, 0002, 0011	—	0002 (Ask-the- graph)	—	0001- 0007	—	0012
UC-FW-04 Export	0001, 0002, 0012	—	—	—	—	0002- 0004	0015
UC-FW-05 Refresh	0001, 0002, 0013	0010	—	—	—	—	0010
UC-FW-06 Investigation	0001, 0002, 0014- 0015	0004	0001- 0007	—	0006	—	0006
UC-FW-07 Audit	0001, 0005, 0012	—	—	0008	—	0005- 0007	0004, 0009

Any requirement without a UC trace is a defect and must be either linked or removed.

6. Out of scope for v1

- Multi-device concurrent drafting.
- Multi-user concurrent review of the same device.
- Web-hosted deployment (v1 is local CLI + local Streamlit).
- Continuous monitoring / auto-refresh on GitHub push.
- Direct FDA submission portal integration.

- SBOM ingestion (v1 reads code + docs; SBOMs are future work).
- Fuzzing or dynamic analysis of firmware (v1 is static-only).

7. Reviewer decisions (locked v1.0)

Question	Decision
OpenAI vs free tier for agents	Hybrid: OpenAI <code>gpt-4o</code> for Threat/Control/Risk only; Groq + Gemini free tier for the other four agents; <code>--profile {free openai hybrid}</code> flag selects at run time
Automatic fallback on free-tier failure	Yes — fall through to OpenAI <code>gpt-4o-mini</code> , recorded in provenance (SRS-AGT-0007)
Latency budget (SRS-NFR-0003)	30 min for 225 threats — Groq is fast enough for 4 of 5 agents
Pinned commit re-clone (SRS-CLI-0005)	Raised to High — audit trail depends on it
Free-tier operable (SRS-NFR-0011)	Raised to High — matches capstone demo story
<code>sra config</code> subcommand	Deferred — manual <code>config.yaml</code> editing is enough for v1

8. Change log

- **v1.0 (2026-09-10)** — Approved. Added SRS-CLI-0016 (`--profile` flag), SRS-AGT-0011 (per-profile model table), reworked SRS-AGT-0007 (cross-provider fallback), reworked SRS-NFR-0002 (per-profile cost caps), reworked SRS-NFR-0005 (OpenAI + Groq + Gemini via LiteLLM), reworked SRS-NFR-0011 (free-tier default demo). Raised SRS-CLI-0005 and SRS-NFR-0011 to High priority.
- **v0.1 (2026-09-10)** — Initial draft. 56 functional + 15 non-functional requirements traced to `docs/use_cases.md` v1.0.